

Guidelines

CSD2401 Technical Design Document Outline

Introduction

This document outlines the requirements for the Technical Design Document (TDD) required for CSD2401. All listed sections must be included, and they must be in the order listed.

While this document is due at the proof-of-concept milestone, you should start working on it asap. This is the document that will tell every programmer on your team what they need to know to really start implementing the game. A good TDD will essentially just describe what you have already done for your engine proof. Most student game engines need only a 15-25 pages TDD at the beginning!

While you should try to think through as much of your technical design as possible in this document, don't worry about getting it perfect or entirely fleshed out. You should cut the design down to what you need for a good first version of your engine. Anything in the architecture that isn't critical to a good first technical engine should be labeled as such (make these things a different color, put them in italics, or whatever works for you).

Some overall guidelines:

- Every page must have a page number (except for the cover page), your game title, and the DigiPen copyright ("All content © 2025 DigiPen (SINGAPORE) Corporation, all rights reserved.").
- Documents must be neatly formatted and easily readable. Put page breaks before new sections (when appropriate), use consistent formatting and fonts, use headings for sections and sub-sections, etc. Sloppy documents are unprofessional and may prevent your project from getting funding.
- Spell-check and grammar-check the document before submission.
- Avoid the use of personal pronouns (I, we, etc.)

When done with your TDD:

- Rename the file to "**CSD2401-TDD-YourTeamName.docx**" before your submission!
- The **technical lead** of the team must submit it onto Moodle. Submission entry links will be provided.

Cover Page

The cover page should contain the following information:

- Game Title
- "Technical Design Document"
- Class name and section (e.g. CSD2401AF25)
- Semester and year (e.g. Fall 2025)
- Team Name
- Team Roster – List all members of the team, including the following information:
 - Student name
 - Official job (or jobs) and Champion positions
 - Coding responsibilities

Table of Contents

The TDD must contain a table of contents (TOC). Make sure the TOC is updated every time the TDD is submitted. If necessary, refer to the Word documentation for help on adding and updating TOCs. The TOC fields must link correctly to their correspondent paragraphs.

CSD2401 Technical Design Document Outline

TDD Structure

Overview

Describe the overall architecture of the project. List and describe each global system your project use (graphics, physics, AI, UI, audio, etc.).

Next, describe your systems and core tools, one by one, as shown below for “Graphics Implementation” paragraph:

Graphics Implementation

What techniques will be used to implement the graphics design? e.g. 2D or 3D or VR or AR..., orthographic or perspective view, OpenGL, or DirectX..., how are you loading assets such as backgrounds, sprites, and animations? What are the rendering capabilities? Any components used? Vertex Format? Rendering pipeline?... [Include some graphical images/flowcharts/screenshots for better explanation]

Add all your other managers/ systems here (like the “Graphics Implementation” section)

** Should have a separate folder named “Data” to place all our source files inside, same hierarchy level as main.cpp*

** One system means ONE folder - any subsequent folders inside this one folder will be up to the one in-charge after getting approval from rachael or if it's nothing major*

Major Systems (Core Engine Components):

1. **[Rachael]** CoreEngine - Application loop, system initialization, shutdown
2. **[PinKai]** Graphics - Rendering pipeline, OpenGL interface, sprite management
3. **[Jun Yong]** Audio - Sound system, beat detection, rhythm synchronization
4. **[Chloe]** Input - Keyboard input, rhythm timing validation
5. **[Chloe & Ben]** Assets - File loading, format parsing for all types of data, data conversion
6. **[Rachael]** GameState - State machine (menu, gameplay, game over)
7. **[Chloe? If noone take i just do roughly ah]** Time - Frame timing, beat tracking, delta time
8. **[Ben]** Math - Mathematical operations, timing calculations
9. **[Reginald & Chloe]** Physic - Basic collision detection for combat
10. **[Jun Yong]** Editor - ImGui level editor, scene editing, asset browser
11. **[Rachael]** ECS - Entity-component management for game objects
12. **[Rachael]** Scene Manager - For use to change scenes(Main menu, levels, death screen)

CSD2401 Technical Design Document Outline

Minor Systems (Game-Specific Components) [Con.t below]:

- 1. Combat System - Weapon logic, combo processing, damage calculation***
- 2. Enemy System - Enemy spawning, basic AI, enemy types***
- 3. Wave System - Wave progression, encounter management***
- 4. Scoring System - Score calculation, streak tracking, combo multipliers***
- 5. UI System - HUD rendering, health display, score display***
- 6. Debug - Debug console, drawing, profiling, logging***

All your other tools here (like “Graphics Implementation” section)

Tools can for example [Math, FPS, Timer, file I/O, memory manager, Images loader, assets manager...] What is their usage for? How do they work? Their format supports?...

Game Object (Entity)

How are your Game objects structured? Depending on if your architecture is Component based or Inheritance based. Do you use Object Factory? Show some examples...

Multiplayer Implementation (if any)

What techniques will be used to implement multiplayer? For single-player games this section can be omitted or contain a sentence explicitly stating that the game is single player.

Coding Methods

Describe all coding conventions that are to be used on the project, including file naming conventions, file locations (i.e., which files go in which folders in the project), code formatting, and code documentation. Also describe what kind of source control system you will use and any rules your team has about its use.

Debugging

Describe the support you have for debugging in your game. Do you have an in-game debug console (or at least a simple output-only console)? Do you have a debug drawing system? Do you have a clean assertion system? Do you have an in-game performance viewer (or profiler)? Do you have a way to watch variables in-game? Can you shut off your debug controls?

CSD2401 Technical Design Document Outline

Tools – Software

Describe any third-party tools used to develop the game. This includes the Microsoft Visual Studio 2022, map editors, art and audio processing tools, testing tools, libraries used and linked to your project etc.

Describe any custom tools that will be developed for the game, including content editors and conversion tools.

Scripting Languages (if any)

Describe any scripting languages that you have created for the game? Which libraries you use? Why did you integrate it in your engine?

Technical Risks

Describe the top two to three technical risks to the project, such as features that might be overly complex, poorly understood, or potentially beyond the capabilities of the platform without additional work. For each technical risk include a plan for mitigating the risk.

CSD2401 Technical Design Document Outline

Appendices

Appendix A: Interface Flow

Flowchart

Create a flowchart that illustrates the entire flow of the interface that is not related to actual gameplay. This should include your intro screen, the main menu, pause screen, setup screens, character or level selection screens, cut-scenes, victory screens, networking screens, etc.

Mockups

Create a mockup and description for each of the screens listed above (this can be very rough hand drawn even). If this seems like too much work, you should probably have a simpler interface and/or a simpler game.

Appendix B: Engine Architecture Flow

Flowchart

Create a flowchart that illustrates the entire engine structure. This flowchart shows the relation between the different components. e.g. it shows the core engine, the systems, the object factory, the winmain (or main) entry point...

You can show more than one flowchart if you want to zoom-in at a specific area in your engine.

Appendix C: Art Requirements

Describe the requirements for all art assets used in the game, including file naming conventions, file format(s), and anything else needed to create the assets.

Describe the source or sources of all the art in your game (programmer art, DigiPen libraries, BFA students, etc.). Remember that you must create all your own art (or use DigiPen libraries). You cannot use your friends, family members, public domain material, etc. You can use art from other students not on your team (but you must give them credit). If you wish to use DigiPen artists, you must talk to your instructor first. You can never use outside artists at all.

Appendix D: Audio Requirements

Describe the requirements for all audio assets used in the game, including file naming conventions, file format(s), and anything else needed to create the assets.

Describe the source or sources of all the audio in your game (programmer audio, DigiPen libraries, other students, etc.). Remember that you must create all your own audio (or use DigiPen libraries). You cannot use your friends, family members, public domain material, etc. You can use audio from other students not on your team (but you must give them credit). You can never use outside musicians at all.

Actual TDD

Pulse Protocol

By Team Bloop

Technical Design Document CSD2401-GAM200 -Fall 2025

<i>Name</i>	<i>Role & Champion</i>	<i>Course</i>
<i>Chia Wei Xuan, Rachael</i>	<i>Technical Lead, Engine Champion</i>	<i>RTIS</i>
<i>GOH Pin Kai</i>	<i>Programmer, Graphics Champion</i>	<i>RTIS</i>
<i>Ban Kai Wei Benjamin</i>	<i>Programmer, Serialization & Asset Champion</i>	<i>RTIS</i>
<i>LAU Chloe Rey En</i>	<i>Product Manager, Input Champion</i>	<i>IMGD</i>
<i>LEU Jun Yong</i>	<i>Programmer, Gameplay Champion</i>	<i>IMGD</i>
<i>Reginald Lew Yee Ren</i>	<i>Programmer, Physics & Collision Champion</i>	<i>IMGD</i>
<i>CHONG Sze Ting Fenny</i>	<i>Art Lead, ART Champion</i>	<i>BFA</i>
<i>Carrie Lam Tze Ying</i>	<i>Design Lead, Design Champion</i>	<i>UXGD</i>

Content Page

1. Overview	4
1.1 Project Brief	4
1.2 Overview	4
1.3 MAJOR SYSTEM (Core Engine Components)	4
2. Entity Component System Architecture	6
2.1 Core ECS Architecture:	6
2.2 How Entity Manager works and how to use it	6
2.3 How Component Manager works and how to use it	6
2.4 How System Manager works and how to use it. And Component/ System Signatures	7
2.5 Why the use a Coordinator	8
2.6 Why the use a Singleton	8
3. Graphics Implementation	9
3.1 Overview	9
3.2 Basic Structure of Graphics Processing	10
3.3 Assets Loading Sequence	12
3.4 Graphics Pipeline	14
3.4.1 Coordinate systems and transforms	14
3.4.2 Camera	15
3.4.3 Batching and draw order	15
3.4.4 Text rendering	16
3.4.5 Debug geometry	17
3.5 Future Implementation	17
4. Audio Implementation	19
Initialization Steps (in AudioManager::Initialize())	19
Update Behavior	19
Shutdown Steps	19
5 Physics/Collision Implementation	20
5.1 Overview	20
5.2 Collision Data: AABB, OBB and Transform	20
5.3 Building AABBs from Transform	21
5.4 AABB and Simple Collision Checks	22
5.4.1 Standard AABB Overlap	22
5.4.2 Edge Contact Detection	22
5.4.3 Distance-Based Collision	22
5.5 Rotated Collision with OBB + SAT (For M2 Rubrics)	23
5.5.1 Building OBB from Transform	23

5.5.2 SAT OBB vs OBB Collision	24
5.5.3 Simple Collision Response	25
6. Factory and Scene Manager	26
6.1 Factory Manager	26
6.2 Scene Manager	26
7. Serialization and Deserialization	28
7.1 Serialization (Saving)	28
How Serialization function Is Used in the Engine	28
7.2 Deserialization (Loading)	29
8. Scripting Languages	30
8.1 Why Csharp language	30
8.2 .NET	30
8.3 How Engine is linked to Csharp	31
9. Technical Risk	34
9.1 Problem 1: C# Scripting Integration Complexity	34
9.1.1 Mitigation	34
9.2 Problem 2: JSON Parsing and Data Validation	34
9.2.1 Mitigation	34
10 Editor UI Implementation	35
10.1 How the Editor Works With the Engine	35
10.2 Scene Viewport	35
10.3 Hierarchy Panel	35
10.4 Inspector Panel	36
10.5 Asset Browser	36
10.6 Gizmo Tools (Move, Scale, Rotate)	37
10.7 Playback Controls (Play, Pause, Stop)	37
10.8 Layer System and Layer Manager	37
11. Coding Methods	38
11.1 Naming Convention	38
11.2 Code Documentation	38
12. Debugging	39
12.1 Assertion System	39
12.2 Visual Studio Debugger Integration	39
12.3 Console Output	39
12.4 ImGui Debug Inspector	40
12.5 Performance Profiling	40
12.6 Debug Control Toggles	40
13. Tools - Software	41
13.1 Development Environment	41
13.2 Graphics and Rendering	41

13.3 User Interface	41
13.4 Data Serialization	41
13.5 Audio	42
13.6 Scripting Runtime	42
Appendix A: Interface Flow	43
Appendix B: Engine Architecture Flow	44
Appendix D: Project dynamic linking diagram	46
Appendix E: Art Requirement	47
Appendix F: Audio Requirements	48

1. Overview

1.1 Project Brief

Pulse Protocol is a custom game engine built using C++ and OpenGL, developed as part of the DigiPen Institute of Technology curriculum. The engine implements an Entity-Component-System (ECS) architecture to provide a flexible, performance-oriented framework for 2D game development. The engine features integrated systems for graphics rendering, audio management, physics simulation, input handling, and scene management for now.

1.2 Overview

The engine architecture I coded is an ECS that uses signatures:

- **Entities** are just IDs that represent game objects.
- **Components** are pure data structures (Transform, AABB, LogicComponent, etc.)
- **Systems** contain logic that operates on entities with specific component signatures.

The **Coordinator** singleton acts as the central mediator, managing entity lifecycles, component registration, and system updates. This design promotes cache-friendly iteration, easy extensibility, and clear separation between data and logic.

1.3 MAJOR SYSTEM (Core Engine Components)

- **[Rachael]** CoreEngine - Application loop, system initialization, shutdown
- **[Rachael]** ECS - Entity-component management for game objects
- **[Rachael]** Factory Manager - Add Entities to a Scene, Save and import entities from a json file.
- **[Rachael]** Scene Manager - For use to save and import scenes from a json file.
- **[Pin Kai]** Graphics - Rendering pipeline, OpenGL interface, sprite management
- **[Jun Yong & Ben]** Audio - Sound system, beat detection, rhythm synchronization
- **[Chloe]** Input - Keyboard input, rhythm timing validation
- **[Chloe & Ben]** Assets - File loading, format parsing for all types of data, data conversion
- **[Chloe]** Time - Frame timing, beat tracking, delta time
- **[Ben]** Math - Mathematical operations, timing calculations
- **[Reginald & Chloe]** Physic - Basic collision detection for combat
- **[Jun Yong]** Editor - ImGui level editor, scene editing, asset browser

1.4 Prerequisites

Visual Studio 2022:

- Download: <https://visualstudio.microsoft.com/downloads/>
- Required workload:
 1. .NET desktop development
 2. Desktop development with C++ (C++/CLI support for v143 build tools (Latest))

CMake (Version 3.15 or higher):

- Download: <https://cmake.org/download/>
- Use the Windows x64 installer (.msi)
- Check "Add CMake to system PATH" during installation

.NET 8.0 Runtime (Required for C# Scripting):

- Go to: <https://dotnet.microsoft.com/en-us/download/dotnet/8.0>
- Scroll to ".NET Runtime 8.0.416" section
- Under Window Binaries column, click "x64"
- Extract the downloaded .zip file
- Go into shared\Microsoft.NETCore.App\8.0.22
- Copy the 184 .dll files and paste them into : BaseEngine/extern/dotnet/bin/
- Important: Do NOT download SDK or Installer - you need "Binaries"!

2. Entity Component System Architecture

2.1 Core ECS Architecture:

Our Engine uses a three manager system that is coordinated by a central Coordinator class. But first let me introduce you to what our ECS system looks like.

Currently, there is:

1. Entity Manager: Create/ Destroys Entities. These Entities are just a number, which are called ID in our Engine.
2. Component Manager: Stores and manages all the components data
3. System Manager: Manages systems and their entities bit signatures.
4. Coordinator: A single Interface that wraps all three managers
5. Singleton: Ensure there is only one instance of Coordinator

2.2 How Entity Manager works and how to use it

The EntityManager is responsible for creating, destroying, and tracking entity IDs. When use use the function CreateEntity(), it adds them into a queue and assigns it an ID. And when you delete them, the ID gets recycled when the max number of ID has been used(currently 5000).

2.3 How Component Manager works and how to use it

The ComponentManager stores all component data and manages component arrays for each component type. In this Manager, the important functions to know are:

1. Register new component types
2. Assigns Entity ID with Component Signatures
3. Creates and manages component arrays (one per component type)
4. Adds/removes/retrieves component data for entities

This means that every new Component will have its own container of component data organized in a packed array, where each component instance is stored contiguously in memory for optimal performance. I will explain more in the signature later below!

2.5 Why the use a Coordinator

The team is using a Coordinator to make life easier. Instead of having to call the individual systems. Just call the Coordinator and have access to all three of the managers. The way to access them is to do a: `auto do = Coordinator::GetInstance();` and then this is how to use it: `do ->CreateEntity();`,

2.6 Why the use a Singleton

The use of a Singleton is to ensure that there would be only one Coordinator instance only. It is being done like this using a Curiously Recurring Template Pattern. This means that when it is called, if has already been instanced, it will point back to that instance of coordinator. And it does this to make sure that multiple separate entity worlds don't run simultaneously(prevents us from accidentally creating a second Coordinator that could fragment our entity data. Because Coordinator is inherited from it, you don't have to call anything new, just the same `auto x = Coordinator::GetInstance();` and thus, can then use x to call any functions in all three manager systems.

3. Graphics Implementation

3.1 Overview

The graphics subsystem is responsible for converting game state (entities and components in the ECS) into 2D images on screen. It targets an OpenGL context and is designed around a fixed virtual resolution of **1920 × 1080**. The actual window size can change, but the game view is letterboxed to preserve aspect ratio.

At a high level the subsystem is split into:

GLHelper: owns the GLFW window and OpenGL context, handles input callbacks and timing, and exposes framebuffer size/letterbox information.

GLApp: low-level rendering backend. It:

- Manages shader programs, vertex/index buffers, and a sprite batcher.
- Converts ECS component data ([Transform](#), [Renderable](#), [Animation](#), [LayerTag](#)) into draw calls.
- Manages a simple 2D camera (pan + zoom) for both the editor and the game.

GraphicsManager: high-level system that integrates graphics into the engine. It:

- Initializes fonts and texture resources from the [AssetRegistry](#).
- Renders all text overlays (UI labels, FPS, debug information).
- Responds to resize events and message callbacks (e.g., toggling debug overlay).

ECS Components: data that feeds the renderer:

- [Transform](#) - position, scale, rotation and the precomputed [mdl_to_ndc_xform](#) matrix.
- [Renderable](#) - which shader/model “slot” to use, texture ID, tint color and sprite name.
- [Animation](#) - sprite sheet layout (rows/columns), frame index and playback speed.

- [TextComponent](#) - string content, font ID, color and visibility flags.
- [LayerTag](#) - layer mask (Background / World / UI), order within layer and z-depth.

DebugDraw - a small utility that renders lines and points (e.g., AABBs) using its own shader and vertex buffers for visual debugging.

Together, these pieces form a data-driven 2D rendering pipeline where the ECS describes *what* to draw and [GLApp/GraphicsManager](#) decide *how* to draw it efficiently.

3.2 Basic Structure of Graphics Processing

The graphics processing follows the engine's main loop and ECS model:

★ Window and context setup (startup)

- [GLHelper::init_glfw\(\)](#) creates the GLFW window and OpenGL context.
- GLEW is initialized and input callbacks are registered.
- [GLApp::init\(\)](#) creates the sprite batcher ([BatchRender](#)), loads all shader programs (background, sprite, font, debug), and prepares a shared VAO/VBO/EBO for quads.

★ Frame preparation

- Each frame, [GLHelper](#) updates [delta_time](#) and polls GLFW events.
- The current framebuffer size is queried; if it changes, [GLApp::OnResize\(\)](#) recomputes the letterboxed game viewport and calls [glViewport\(\)](#) with the correct offset and size.

★ ECS transform update

- [Transform::update\(delta_time, entitySet\)](#) iterates over all entities that have a [Transform](#) component.
- For each entity, it composes a 3×3 transform matrix based on position, scale and rotation, then multiplied by a [ScreenToNDC](#) matrix derived from the virtual resolution.

- The result is stored in `Transform::mdl_to_ndc_xform`, which maps from model space directly to NDC.

★ World rendering

- `GLApp::draw(entities, includeMask)` is called with the set of entities to render and a layer filter.
- `GLApp`:
 - Filters entities that have both `Transform` and `Renderable` (and optionally `Animation` / `LayerTag`).
 - Ensures each `Renderable` has a valid `textureID`, lazily loading textures via the asset system if needed.
 - Collects visible entities into a `drawList`.

★ Sorting and batching

- The `drawList` is **stable-sorted** using `LayerTag`:
 - First by layer (`Background < World < UI`),
 - then by `orderInLayer`,
 - then by `z` depth.
- For each entity, `GLApp` computes per-sprite UVs from its `Animation` data and pushes the quad into the `BatchRender` using a key (`shader index, model index, texture ID`).

★ Issuing GPU draws

- `BatchRender::Flush()` walks all buckets (grouped by key), binds the correct shader and texture, uploads the packed vertices into the shared VBO, and calls `glDrawElements()` once per bucket.
- Vertices are pre-transformed to NDC on the CPU, so the sprite shaders use an identity `uModel_to_NDC` matrix and just sample the texture.

★ Text and debug overlays

- After the world is drawn, `GraphicsManager::Update()` renders text components using a separate VAO/VBO and orthographic `projection` matrix.
- `DebugDraw::Begin()/End()` optionally draw line primitives (e.g., AABBs) on top of the scene using their own shader.

★ Present

- Finally, GLFW swaps the back buffer to display the completed frame.

This structure cleanly separates *simulation* (ECS updates) from *rendering* (GLApp/GraphicsManager) while minimizing OpenGL state changes via batching.

3.3 Assets Loading Sequence

The graphics subsystem relies on an `AssetRegistry` that describes all textures and fonts used by the game. The loading sequence is:

★ Asset registry setup

- During engine bootstrap, the `AssetRegistry` is populated from configuration/JSON files with records such as:
 - Texture ID $\Rightarrow$ file path
 - Font ID $\Rightarrow$ font file path and size
- `GraphicsManager` is constructed with a pointer to this registry (`AssetRegistry *registry`).

★ Texture loading

Preloading from registry

- `GraphicsManager::LoadTexturesFromRegistry()` reads the texture container from the registry.
- For each texture record:
 - The file path is passed to `GraphicsManager::LoadTextureFile(path)`.

- `stb_image` ([stb_image.h](#)) decodes the image into RGBA8 pixel data.
- `upload_rgba8()` creates an OpenGL 2D texture ([glGenTextures](#), [glTexImage2D](#), mipmap generation, filtering parameters).
- Two maps are updated:
 - `texHandle_key`: `std::string` → `GLuint` (from asset name to texture handle),
 - `texHandle_id`: `GLuint` → `std::string` (reverse lookup, used for debugging / tools).

★ On-demand loading

- For [Renderable](#) components that reference a sprite by name, if `textureID` is 0 at draw time, [GLApp](#) or [GraphicsManager](#) calls `GetOrLoad(name)`:
 - If the texture is already in `texHandle_key`, its currently loaded handle is returned.
 - Otherwise [LoadTextureFile](#) is invoked and the maps are updated.

★ Utility textures

- [InitWhiteTexture\(\)](#) creates a 1×1 white texture used as a fallback for UI / tinted quads when no explicit texture is bound.

★ Font loading

- [GraphicsManager::InitFonts\(\)](#) initializes the FreeType library ([FT_Init_FreeType](#)) and then calls [LoadFontsFromRegistry\(\)](#).
- For each font entry in the registry, [LoadFontFile\(fontPath, fontKey, pixelHeight\)](#):
 - Loads the font face with FreeType ([FT_New_Face](#)).
 - Sets the pixel size and loops over the supported glyph range (typically ASCII).
 - For each character:
 - FreeType rasterizes the glyph to a grayscale bitmap.
 - An OpenGL texture is created for that glyph and its metrics are recorded:

- size, bearing, and advance.
- A `Character` struct is stored in the font's `FontData::glyphs` map.
- The `FontData` is inserted into `m_fontMap` under the given `fontKey`.
- Once fonts are loaded, `GM_FindFont(fontId)` provides const access to the corresponding `FontData` for text rendering.

This asset loading design allows most content to be preloaded from the registry for fast startup while still supporting on-demand loading when new textures are referenced at runtime.

3.4 Graphics Pipeline

The concrete rendering pipeline is built around a CPU-side transform to NDC, a simple 2D camera, and a batched quad renderer.

3.4.1 Coordinate systems and transforms

★ Model space

- Sprites are defined as unit quads centered at the origin:
 - Background (`mdl_ref == 0`): quad from $(-1, -1)$ to $(1, 1)$ to cover the full game view.
 - Normal sprites: quad from $(-0.5, -0.5)$ to $(0.5, 0.5)$.

★ World / screen space

- `Transform` stores `Pos` (in pixels relative to the virtual screen), `Scale` (in pixels), and `Rotation`.
- `Transform::update` builds a 3×3 matrix `mdl_to_ndc_xform`:
 - Scale matrix from sprite size,
 - Rotation matrix about the origin,
 - Translation matrix from `Pos`,
 - A `ScreenToNDC` matrix based on `VIRTUAL_W` and `VIRTUAL_H`:

All content © 2025 DigiPen (SINGAPORE) Corporation, all rights reserved.

3.4.2 Camera

★ `GLApp` defines a `Camera2D` struct with:

- `center` - camera pan in NDC space,
- `zoom` - scaling factor with clamped `minZoom` / `maxZoom`.

★ `BuildPanZoomNDC(cam)` constructs a 3×3 matrix $\text{ZoomNDC} = T * S$ that first scales by `zoom` and then translates by `center`.

★ At draw time, for each sprite:

- `M = mdl_to_ndc_xform` is converted to `glm::mat3`.
- The final transform is $\text{modelNDC} = \text{ZoomNDC} * M$.
- This matrix is applied on the CPU when generating the positions sent to the VBO.

Two cameras exist: `gEditorCamera` for editor view and `gGameCamera` for gameplay, but only one is active at a time.

3.4.3 Batching and draw order

★ `BatchRender::Push()`:

- Computes the four model-space corners for the quad (background vs sprite).
- Transform each corner with `modelNDC` to get final NDC positions.
- Packs positions and UVs into a `Vtx` list.
- Chooses a bucket key (`shd_ref`, `mdl_ref`, `textureID`) and appends 4 vertices to that bucket.

★ `BatchRender::Flush()`:

- Copies each bucket's vertices into a dynamic VBO using `glNamedBufferSubData`.

- Binds the shared VAO and the bucket's texture.
- Uses `GLApp::shdrpgms[shd_ref]` as the active shader program. Sprite and background shaders (`sprite_vs/fs`, `bg_vs/fs`) share the same vertex layout.
- Sets:
 - `uModel_to_NDC` = identity matrix (vertices are already in NDC),
 - `uOffset` and `uScale` = (0, 0) and (1, 1) by default, since UVs are precomputed.
- Issues a single `glDrawElements(GL_TRIANGLES, spriteCount * 6, ...)` per bucket.

Because entities are pre-sorted using `LayerTag`, background elements are always drawn first, followed by world sprites, and finally UI elements.

3.4.4 Text rendering

Text is rendered after the sprite batch using a separate pipeline:

- ★ A dedicated VAO/VBO stores glyph quads in screen space.
- ★ An orthographic `projection` (0..VIRTUAL_W, 0..VIRTUAL_H) is used in `font_vs.vert`.
- ★ `GraphicsManager::DrawText2D()`:
 - Look up `FontData` via `GM_FindFont(fontId)`.
 - Iterates characters, retrieving `Character` glyph data (texture, size, bearing, advance).
 - For each glyph, computes a quad at the desired pixel position and uploads it into the glyph VBO.
 - Uses `font_fs.frag`, which samples the glyph mask from the red channel and multiplies it by `textColor`. Alpha blending is enabled (`GL_SRC_ALPHA`, `GL_ONE_MINUS_SRC_ALPHA`).

`Update2DTextComponent()` bridges ECS and rendering:

- For each `TextComponent`, it computes the correct screen-space position (either absolute screen position for debug labels, or derived from world position + offset for normal entities).

- If `visible` is true, it calls `DrawText2D()` with the component's text, font, color and size.

3.4.5 Debug geometry

`DebugDraw` exposes `Begin()`, `DrawPoint()`, `DrawLine()`, `DrawRect()`, etc.

Geometry is accumulated into CPU arrays and then flushed once per frame using a simple shader pair:

- ★ A vertex shader that transforms 2D positions by an MVP matrix (`uMVP`),
- ★ A fragment shader that outputs a solid color.

`GLApp::DrawAABB_NDC()` converts a world-space AABB into NDC using `mdl_to_ndc_xform` and emits four lines via `DebugDraw`. This is mainly used to visualize collision bounds and layout.

3.5 Future Implementation

The current graphics subsystem is functional for a 2D sprite-based game with text overlays and debug drawing. Several extensions are planned or made possible by the current design:

★ Expanded layering and render passes

- Add more logical layers (e.g., “ParallaxBackground”, “ForegroundEffects”, “Debug”) without changing the core pipeline, by extending the `LayerMask` enum and the `LayerTag` component.
- Introduce additional render passes for effects such as bloom or vignette by drawing to off-screen framebuffers before presenting to the main framebuffer.

★ GPU-driven transforms

- Move from CPU-side NDC generation to GPU-side matrix multiplication:
 - Store model matrices per instance and send them as instance attributes or uniform buffers.
 - This would reduce CPU cost for large numbers of sprites and make it easier to add camera effects and per-object animation on the GPU.

★ Texture atlases and material system

- Formalize sprite sheets and atlases as first-class “materials” instead of manually tracking [rows](#), [columns](#) and sprites per [Renderable](#).
- Allow material properties (e.g., tint, blend mode, shader index) to be edited in the editor and reused across entities.

★ Post-processing and screen effects

- Introduce an off-screen render target for the main world pass, then apply post-processing shaders (e.g., color grading, screen-shake, vignette) before compositing UI.

★ Improved debug and profiling tools

- Extend [DebugDraw](#) and the debug overlay to visualize:
 - Layer boundaries, camera frustum, tile grids.
 - Per-frame batch statistics (number of sprites, draw calls per bucket, texture switches).
- Integrate real-time metrics into the FPS/debug text entities for optimization.

★ Editor integration

- Expose camera parameters, layer masks, and renderable properties directly in the editor’s panels.
- Allow live reloading of shaders and textures from disk to speed up iteration.

These future steps build on the existing architecture (ECS-driven components + GLApp batch renderer + GraphicsManager overlays), so they can be implemented incrementally without a major rewrite of the graphics core.

4. Audio Implementation

The audio subsystem is implemented using the FMOD library. In an **AudioManager** object with type class.

The variable **mSystem** is a pointer to the **FMOD::System**.

Initialization is performed inside **AudioManager::Initialize()**, which sets up the FMOD system, mixer groups, and loads audio metadata from the Asset Registry.

Initialization Steps (in AudioManager::Initialize())

1. Ensure Asset Registry exists and assigned to the **AudioManager**
2. Create FMOD system Object
 - **mSystem**
3. Initialize FMOD
4. Retrieve the Master Channel Group
5. Create Submix groups
 - Music Group
 - SFX Group
6. Attach Submix Groups to Master
7. Apply Volume Settings
 - For each channel (Master, Music, SFX)
8. Load Audio metadata from Registry
 - By Functions **LoadAllBGM** and **LoadAllSFX**
 - Audio metadata are located in the audioResources container found in AssetRegistry
9. Log message on console confirming Initialization is a success

The audio update loop is executed every frame through **AudioManager::Update()**.

Update Behavior

1. Check FMOD System status
2. Check Engine Play State
3. Process Audio Components if system exists and engine is in play state
4. Then update **mSystem** (FMOD system)

The audio system is cleaned up through **AudioManager::Shutdown()** and **releaseAll()**.

Shutdown Steps

1. Stop All Audio playbacks
2. Clear Active Channels
3. Release FMOD::Sounds
4. Close and Release FMOD System
5. Log message on console confirming shutdown is a success

5 Physics/Collision Implementation

5.1 Overview

Our physics layer is intentionally lightweight and focused on **2D collision detection**. The goal is to provide:

- Consistent hit detection for combat (player vs enemies, hitboxes, etc.)
- Simple spatial queries for GUI and mouse interaction
- A clear separation between **collision math** and **gameplay scripts**

All collision-related logic is centralized in the **CollisionManager** system. Gameplay scripts (e.g., enemy / player scripts) are responsible for deciding **what to do** when a collision occurs, while **CollisionManager** is responsible for **figuring out if two things are colliding** using AABB.

5.2 Collision Data: AABB, OBB and Transform

Collision in Pulse Protocol is entirely component-driven:

- **Transform (Framework::Transform)**
Stores position, scale, and rotation for each entity in pixel space. This is the single source of truth for where an object is in the world.
- **AABB (Axis-Aligned Bounding Box)**
Used as the primary collision primitive for most interactions. It is defined by:
 - **min** and **max** corners (world-space)
 - **width** and **height** of the box
- **OBB (Oriented Bounding Box) (Did for M2 rubrics and future purposes if needed)**
Used for more accurate collision around rotated objects (e.g., slanted hitboxes, rotated enemies). The **OBB** struct stores: center, half-extents, rotation in degrees, and two local axes used in the SAT (Separating Axis Theorem) test.
collision

The collision system converts **Transform** data into AABBs and OBBs every frame when needed, so the game code never manually computes boxes. This keeps all collision math consistent and fully centralized.

5.3 Building AABBs from Transform

To avoid hard-coding collision sizes everywhere, we use a helper function:

- `AABB BuildAABBFromTransform(const Framework::Transform& transform, const AABB& baseBox)`

This function takes a **base AABB** (usually defined in prefab data or as a logical “unit” box) and scales it based on the entity’s current `Transform.Scale`. Then it positions the box so that it is centered on `Transform.Pos`.

Key details:

- The **width** and **height** of the final AABB are derived from:
 - `baseBox.width * transform.Scale.x`
 - `baseBox.height * transform.Scale.y`
- `min` and `max` are then computed as:
 - `min = center - (width/2, height/2)`
 - `max = center + (width/2, height/2)`

This means:

- Designers or programmers only need to define **one logical collision box** per entity type.
- The same base box automatically adapts when the entity is scaled in the editor or during gameplay.
- All AABB calculations stay in world space, which makes downstream checks simpler.

5.4 AABB and Simple Collision Checks

For most of our game, we rely on **fast, axis-aligned collision checks**.

5.4.1 Standard AABB Overlap

- `bool CheckAABBCollision(const AABB& a, const AABB& b)`

This is a classic AABB overlap test. It returns `true` when the projections of both boxes overlap along both X and Y axes. It is used for general-purpose “is overlapping?” checks such as: player vs enemy hurtbox, enemy vs attack range, or area triggers.

5.4.2 Edge Contact Detection

- `bool CheckEdgeContactCollision(const AABB& a, const AABB& b, float threshold)`

In rhythm combat, we sometimes care about **edge contact** rather than large overlaps (e.g., “attack just touching the enemy on beat”). This helper:

- Detects when the **sides of two AABBs are close within a configurable threshold** on X or Y.
- Also checks that the centers of the boxes are reasonably aligned on the other axis so that small height/width differences don’t prevent a valid edge contact.

5.4.3 Distance-Based Collision

- `bool CheckDistanceCollision(const Vector2& a, float aradius, const Vector2& b, float bradius)`

For circular or radial checks (e.g., radial hit area, proximity checks), we use a simple distance test:

- Computes distance between two positions `a` and `b`.
 - Compares it to the sum of their radii: collision occurs if `dist < (aradius + bradius)`.
- collision
]

This is useful when we don’t want to define full boxes (for example, “within N pixels of this point”).

5.5 Rotated Collision with OBB + SAT (For M2 Rubrics)

Some entities in Pulse Protocol rotate (or we want rotated hitboxes for more precise combat). For that we use **Oriented Bounding Boxes (OBB)** and the **Separating Axis Theorem (SAT)**.

5.5.1 Building OBB from Transform

- **OBB** `BuildOBBFromTransform(const Framework::Transform& transform, const AABB& baseBox)`

This function converts a `Transform` + base AABB into an OBB:

collision

1. Center

- Set directly to `transform.Pos`.

2. Half-extents

- Internally calls `BuildAABBFromTransform` and uses half of the AABB width/height as the OBB half-extents. This keeps the OBB size consistent with our existing AABB sizing rules.

3. Rotation

- Reads `transform.Rotation.x` (in degrees) as the box rotation.
- Converts degrees to radians internally for trigonometric functions.

4. Local axes

- Computes two perpendicular unit vectors:
 - Axis 0 = rotated X axis
 - Axis 1 = rotated Y axis

These axes are later used as SAT test axes and to derive the rotated corners of the OBB.

5.5.2 SAT OBB vs OBB Collision

- `bool CheckOBBOvsOBB(const OBB& obbA, const OBB& obbB)`

This function implements a **full SAT test** between two OBBs. High-level flow:

1. Axis generation

- Builds 4 axes:
 - 2 from OBB A (its local X and Y)
 - 2 from OBB B
- Each axis is normalized before use.

2. Projection

- For each axis:
 - Computes all 4 world-space corners of each OBB using its center, half-extents, and rotation.
 - Projects those corners onto the axis to find `min` and `max` scalar intervals.

3. Separation check

- If on any axis the intervals **do not overlap** (beyond a small epsilon), SAT says the boxes do not collide and we can early-out.

4. Epsilon tolerance

- We use a small `SAT_EPS` to tolerate tiny gaps caused by floating-point precision, so near-edge contacts still count as collisions.

If no separating axis is found, the function returns `true` and we treat the two OBBs as colliding. The function also includes detailed debug prints (axes, projections, corners) to help us tune collision behaviour during development.

5.5.3 Simple Collision Response

- `void ResolveOBBCollision(Framework::Transform& a, Framework::Transform& b, float elasticity = 0.25f)`

Our project does not implement full physics with velocities and impulses. Instead, we provide a **simple positional correction**:

- Compute a normalized direction from `a` to `b`.
- Move each object **slightly apart** along that direction by a factor scaled with `elasticity`.
collision

This is enough to:

- Prevent objects from visually interpenetrating after a collision.
- Give a basic “push-apart” effect without requiring a full physics solver.

Actual game logic (e.g., damage, stun, combo reset) is handled in gameplay scripts when `CheckOBBvsOBB` or `AABB` tests return true.

6. Factory and Scene Manager

The Factory Manager and Scene Manager work together to provide high-level entity creation and scene persistence. While both systems deal with entity instantiation and JSON serialization, they serve different purposes and operate at different scopes.

6.1 Factory Manager

The Factory Manager handles **individual entity creation** from prefab templates. A prefab is a reusable entity archetype defined in JSON that specifies which components an entity should have and their initial values.

Purpose and Use Cases:

The Factory Manager is primarily designed for use within the ImGui editor during level design. Designers can:

- Select a prefab from the asset browser (e.g., "Enemy_Basic", "Collectible_Coin", "Obstacle_Spike")
- Click in the scene viewport to spawn an instance
- Modify component values in the inspector for that specific instance
- Spawn multiple entities from the same prefab with different positions or properties

Factory Workflow:

1. Load Prefab: Factory Manager reads the JSON file and caches the component data.
2. Instantiate: When spawning, the Factory creates a new entity via the Coordinator.
3. Add Components: For each component in the prefab definition, the Factory adds it to the entity with the specified values.
4. Position Override: The spawn position can be overridden (e.g., mouse click location in editor).
5. Return Entity ID: The newly created entity ID is returned for further manipulation if needed.

6.2 Scene Manager

The Scene Manager handles entire scene serialization and deserialization. A scene represents a complete game state including all entities, their components, and scene-specific configuration.

Purpose and Use Cases:

The Scene Manager is used for:

- Saving complete levels - Designers build a level in the editor, then save it as a scene file
- Loading game states - Loading MainMenu.json, Level1.json, BossRoom.json, etc.

- Scene transitions - Switching from gameplay to death screen to main menu
- Persistence - Saving the current game state to resume later (checkpoint system)

Scene Manager Workflow:

Saving a Scene:

1. Iterate All Entities: Scene Manager queries the Coordinator for all active entities.
2. Serialize Components: For each entity, iterate through all its components and extract their current values.
3. Build JSON: Construct the complete scene JSON structure with all entity data.
4. Write File: Save the JSON to disk (e.g., "Level1.json" in assets/scenes/).

Loading a Scene:

1. Clear Current Scene: Destroy all existing entities to start fresh.
2. Parse JSON: Read and validate the scene file using RapidJSON.
3. Create Entities: For each entity in the JSON, create a new entity via the Coordinator.
4. Add Components: For each component listed, add it to the entity with the stored values.
5. Restore State: Scene configuration (background, music, etc.) is applied.
6. Initialize Systems: All systems automatically detect the new entities via signature matching.

7. Serialization and Deserialization

Serialization and deserialization are used to convert data between C++ objects and JSON files. Our engine uses the RapidJSON library to read (deserialize) JSON files into C++ structures during loading, and to write (serialize) C++ data back into JSON when saving scenes, entities, project assets and settings (config). This system allows designers to edit game data externally and ensures that engine state can be stored persistently across sessions. Because assets and settings are defined in JSON rather than hard-coded in C++, new assets can be added or modified without requiring recompilation of the C++ codebase.

7.1 Serialization (Saving)

C++ objects that support serialization implement a **Serialize** member function within their struct or class.

This function accepts:

- a **rapidjson::Value&** representing the JSON object to write into, and
- a **rapidjson::Document::AllocatorType&** used to allocate JSON fields.

Each component or data structure writes its internal state (e.g., position, volume, color, WindowWidth) into the provided JSON object. This ensures that the scene, entities, assets and settings can be saved consistently and reconstructed later exactly as they were.

How Serialization function Is Used in the Engine

The Serialization function is called in the Saving functions . Examples include:

- **Scene Saving**
The **SceneManager** iterates through all active entities in the ECS.
For each entity, it serializes all its attached components (**Transform**, **Renderable**, **AudioSource**, **Animation**, etc.) into a JSON array.
- **Prefab Generation**
When a user creates an entity in the editor, the engine allows the entity to be by calling **SaveEntityAsPrefab** which would call **SaveEntityToJson** saving all the components they contain and the data inside the components . This prefab can be **Deserialized(loaded)** later that will be recreated with its exact name, components and component data. Enabling reusability

Basically any engine object that needs to be saved provides a Save function, and the data it contains is written out using its Serialize function. This modular structure keeps serialization isolated within each component or data object, while higher-level systems manage writing the final JSON file to the desired folder using `std::filesystem`.

7.2 Deserialization (Loading)

C++ objects that can be reconstructed from JSON implement a **Deserialize** member function.

This function accepts a

- **rapidjson::Value const &** representing the JSON object to read from.

During deserialization, the following checks are performed:

1. **Verify that the JSON object exists**
(value.IsObject())
2. **Check that required fields are present**
(value.HasMember("fieldName"))
3. **Validate the datatype of each field**
(value["x"].IsFloat(), value["name"].IsString(), etc.)
4. **Assign values to the corresponding C++ data members**
Only after all validation checks pass.

These checks help prevent invalid JSON data from being assigned to components. Deserialization is used when loading scenes, prefabs, assets, window configuration and editor configuration files.

For any engine object that supports loading, a corresponding **Load** function is provided. This function is responsible for opening the JSON file via **RapidJSON's IStreamWrapper** and **Document** Object. Then calling the object's **Deserialize** function to construct its data members (e.g. audio_id , path, Position).

8. Scripting Languages

8.1 Why Csharp language

The team chose C# as our scripting language for Pulse Protocol for its ease of use for our UXGD Designers to code on, as is similar enough that if she needs help, us IMGD and RTIS can adapt quickly and help out.

8.2 .NET

Our engine is written in C++, but the team wanted to run C# scripts . The .NET runtime acts as the **bridge** that allows our C++ engine to execute C# code.

Key .NET Components are being used:

1. .NET 8.0 Runtime

2. CoreCLR Host API

- C++ API that lets us embed the .NET runtime into our engine
- Found in coreclrhost.h
- Allows C++ code to start the runtime, load assemblies, and call C# methods

3. Common Language Runtime (CLR)

- The component that actually runs the IL code
- Manages memory, threads, and security
- Provides services like exception handling

8.3 How Engine is linked to Csharp

Linking C# to our C++ engine required solving a fundamental problem: C++ and C# are incompatible languages that cannot directly communicate with each other. They use different memory models, different type systems, and different calling conventions. Thus, the need for a bridge.

1. Tier 1: MyEngine (Pure C++) This is our engine core - the ECS, rendering system, physics, audio, everything. It's written entirely in C++ and compiled as a DLL. This layer has no knowledge of C# whatsoever. It simply exposes certain functions that can be called from outside the DLL.
2. Tier 2: ScriptAPI (C++/CLI Bridge) This is the magic layer. C++/CLI is a special Microsoft extension to C++ that can understand and speak to both C++ and C# simultaneously. Think of it as a bilingual translator sitting between two people who speak different languages. On one side, it can call pure C++ functions from MyEngine. On the other side, it exposes C#-compatible classes and methods that ManagedScripts can use.
3. Tier 3: ManagedScripts (Pure C#) This is where game designers write gameplay code in pure C#. They have no idea that C++ exists underneath - they just use clean C# classes and methods that feel natural. When they call a method, it secretly goes through the ScriptAPI bridge down to the C++ engine.

How The Bridge Works

The connection happens through a process called interop (interoperability).

When you write a C# script and call something like `entity.SetPosition(100, 200)`, here's the journey that call takes:

Step 1: C# Method Call Your pure C# script calls a method on what appears to be a C# object. This object lives in the ScriptAPI layer.

Step 2: C++/CLI Translation The ScriptAPI receives this call in its C++/CLI code. Remember, C++/CLI can understand both languages. It takes the C# data types (managed types like `String^`, garbage-collected objects) and converts them into C++ data types (raw pointers, primitive values).

Step 3: Native Function Call Once the data is converted to C++ format, the ScriptAPI calls the actual C++ function in MyEngine. This might be something like updating a Transform component in the ECS.

Step 4: Engine Executes MyEngine receives the call just like any normal C++ function call, processes it (updates component data, runs physics, whatever), and potentially returns a result.

Step 5: Return Journey If there's a return value, it travels back up the chain - C++ returns to C++/CLI, which converts it to C# types, which gets returned to your script.

All of this happens transparently. From the game designer's perspective, they're just calling C# methods on C# objects. They don't see the bridge.

What Actually Happens:

In the C# Layer (ManagedScripts): The script calls `GetPosition()` on its entity object. This entity object is actually a C# wrapper class defined in ScriptAPI.

In the Bridge Layer (ScriptAPI): The C# entity wrapper receives the `GetPosition()` call. Inside this wrapper is a stored integer - the native entity ID from the C++ engine. The wrapper takes this ID and calls a native C++ function, passing the ID as a parameter.

In the Engine Layer (MyEngine): The C++ function receives the entity ID, uses it to look up the entity in the ECS Coordinator, gets the Transform component, reads the position values, and returns them.

Back Up Through The Bridge: The C++ position data (two float values) travel back to ScriptAPI. The C++/CLI code wraps these floats in a C# Vector2 object and returns it.

Back to C#: The script now has a Vector2 with the position. It modifies the values, then calls `SetPosition()` with the new values.

Down Through The Bridge Again: `SetPosition` in the C# wrapper extracts the float values from the Vector2, calls the native C++ `SetPosition` function with the entity ID and the new float values.

Engine Updates: The C++ engine receives the new position values, looks up the entity's Transform component, and updates the position data in the ECS.

It's all a lil wordy, but if you are a IMGD or UXGD, imo, you do not need to know this. You can just do the scripting in Csharp without understanding this.

9. Technical Risk

9.1 Problem 1: C# Scripting Integration Complexity

Integrating C# scripting into the C++ engine requires a three-tier architecture (MyEngine C++, ScriptAPI C++/CLI bridge, ManagedScripts C#) with .NET runtime hosting. The C++/CLI bridge layer is particularly complex as it must translate between incompatible type systems and memory models. Issues include: managing object lifetimes across the native/managed boundary, preventing memory leaks when .NET garbage collector runs, handling exceptions that cross between C++ and C#, and maintaining correct entity references when C# holds pointers to C++ objects. Additionally, C++/CLI has strict limitations on what C++ code it can compile

9.1.1 Mitigation

Ask TA for help.

Implement things step by step. And if there is any errors please seek help from tech lead(did the scripting)

9.2 Problem 2: JSON Parsing and Data Validation

Scene files, entity prefabs, and configuration data are stored as JSON for human readability and easy editing. However, JSON is error-prone when manually edited. Designers may introduce syntax errors (missing commas, unmatched braces, incorrect indentation), type mismatches (string where number expected, boolean as string), or missing required fields (entity without Transform component, prefab missing texture path). RapidJSON will fail to parse malformed JSON, but error messages may be cryptic (line/column numbers without context) or misleading. Silent failures are also possible where JSON parses successfully but contains incorrect data that causes runtime errors later (wrong entity ID references, invalid file paths, out-of-range values).

9.2.1 Mitigation

Implement comprehensive JSON validation layer that checks: syntax validity before parsing, required fields presence for each object type, data type correctness (numbers are numbers, strings are strings), value range validation (entity IDs exist, file paths point to real files, health values are positive).

10 Editor UI Implementation

10.1 How the Editor Works With the Engine

The Editor is connected to the CoreEngine using a registered frame callback.

Every frame, the engine:

1. Updates ECS systems
2. Renders gameplay
3. Calls the Editor callback
4. The Editor draws ImGui UI on top of the framebuffer

Because the editor is compiled as a separate project, it never modifies the engine's internal systems. Switching between Editor Mode and Game Mode is done through the build scripts: **run_editor.bat** includes the editor tools, while **run_game.bat** launches the engine without them. This keeps gameplay logic and rendering unaffected, and both modes share the same CoreEngine runtime to ensure consistent behavior.

10.2 Scene Viewport

The Scene Viewport displays the game's real framebuffer inside an ImGui window.

It uses a dedicated framebuffer, so resizing or docking editor panels does not distort the final render.

Core features include:

- Click-to-select entities
- Camera panning and zooming
- Accurate rendering of sprites, UI, and gameplay elements
- Real-time updates to reflect ECS changes

The viewport acts as the primary visual workspace for building and editing levels.

10.3 Hierarchy Panel

The Hierarchy lists all active entities in the current scene.

It provides a structured way to browse and manage entities, which internally are only numeric ECS IDs.

Features included:

- Create new entities
- Delete entities (via button or the Delete key)

All content © 2025 DigiPen (SINGAPORE) Corporation, all rights reserved.

- Drag-and-drop reordering within the list
- Prefab instance indication
- Non-movable debug objects (entities beginning with _)
- Selection highlight that syncs with the Scene Viewport

The Hierarchy also supports **Undo**, allowing users to revert recent actions such as entity creation, deletion, and transform modifications (up to 20 steps). This ensures safer editing and prevents accidental changes during level construction.

10.4 Inspector Panel

The Inspector provides a structured view of components for the selected entity.

Any change in the Inspector updates the ECS immediately and is reflected in the viewport the next frame.

Supported editing includes:

- Name and visibility
- Transform (position, scale, rotation)
- Renderable / texture assignment
- Animation settings
- Audio properties (volume, loop)
- Attached scripts
- Text component editing (font, content, size)
- Layer assignment

Prefab-specific options include:

- “Linked to Prefab” indicator
- “Apply to Prefab” to push changes back to the source prefab file

The Inspector is the main tool for adjusting gameplay data visually without modifying JSON.

10.5 Asset Browser

The Asset Browser shows all project assets loaded by the engine: textures, sounds, fonts, and prefabs.

It is fully integrated with the AssetRegistry and AssetsManager, meaning new files appear automatically after a refresh.

Supported features:

- Drag-and-drop textures onto entities or components
- Drag files from Windows Explorer into folders to import assets
- Thumbnail size adjustment
- Prefabs: click to instantiate directly into the scene

- Hot-reloading support for textures and audio on disk changes

This system removes the need to edit JSON manually and provides a visual, designer-friendly workflow.

10.6 Gizmo Tools (Move, Scale, Rotate)

The Editor includes a transform gizmo that allows designers to manipulate entities directly inside the Scene Viewport. It supports three modes, Move, Scale, and Rotate similar to those found in commercial game engines.

- **Move** allows repositioning along the X/Y axes.
- **Scale** adjusts size either uniformly or per-axis.
- **Rotate** uses a circular rotation handle for angle control.

The gizmo updates the entity's Transform component instantly, allowing precise and intuitive spatial editing without manually typing numeric values.

10.7 Playback Controls (Play, Pause, Stop)

The Editor includes integrated Play Mode controls in the top bar:

- **Play:** Start runtime simulation while keeping editor UI visible
- **Pause:** Freeze the game state
- **Stop:** End simulation and return to edit mode
- **Resume:** Continue from a paused state

Play Mode uses the same CoreEngine loop, ensuring reliable testing of gameplay systems inside the editor.

10.8 Layer System and Layer Manager

Entities can be assigned to three primary layers:

- **Background**
- **World**
- **UI**

The Layer Manager allows:

- Showing/hiding entire layers
- Locking layers to prevent accidental editing
- Viewing total entity count per layer
- Quick actions: Show All / Hide All

The layer system improves scene organization and prevents unintended edits.

All content © 2025 DigiPen (SINGAPORE) Corporation, all rights reserved.

11. Coding Methods

11.1 Naming Convention

For the most part, the team is using cap for the first letter of the word. Example: CreateEntity, GetSystem, GrabComponent.

However, it is also okay to use small for the first word, then cap for the next. Example: addScript, deleteScene

And if the word is too long, the use of _ is okay. Example: Counter_For_Scene_Entities.

11.2 Code Documentation

Once you are done with a file, please include:

1. Name of the file
2. Project name
3. Author with % of how much work you did
4. Brief
5. Copyright: Copyright (C) 2025 DigiPen Institute of Technology. Reproduction or disclosure of this file or its contents without the prior written consent of DigiPen Institute of Technology is prohibited.

Example:

```

/*****
**
* @file      CoreEngine.cpp
* @project   Pulse Protocol
* @author    Chia Wei Xuan Rachael (primary) - 50%
* @author    Chloe, Ben, Reginald, Pin Kai, Jun Yong (secondary) - 10% each
* @brief     Implementation of the core engine class that initializes all
*            engine subsystems (graphics, input, audio, physics), manages
*            the main game loop with delta time calculation, handles system
*            updates, integrates ImGui editor support, and provides debug
*            rendering utilities for development visualization.
*
* @copyright Copyright (C) 2025 DigiPen Institute of Technology.
*            Reproduction or disclosure of this file or its contents without the
*            prior written consent of DigiPen Institute of Technology is prohibit
*/
*****/
```

12. Debugging

Pulse Protocol includes several debugging systems to aid development and troubleshooting.

12.1 Assertion System

The engine implements a comprehensive assertion system that validates critical operations at runtime. Assertions catch common errors during development.

ECS Assertions: Attempting to add a component to a non-existent entity, trying to retrieve a component an entity doesn't have, exceeding maximum entity limit (5000 entities), registering more than the maximum number of component types, using an unregistered component or system.

File I/O Assertions: Missing asset files (textures, audio, JSON files), invalid file paths, corrupted or malformed JSON data, missing required JSON fields in prefabs or scenes.

When an assertion fails, the engine halts execution and displays an error message indicating the exact failure point, making debugging significantly faster.

12.2 Visual Studio Debugger Integration

The engine is fully compatible with Visual Studio's debugging tools. Breakpoints can be set anywhere in engine or game code. Execution pauses at breakpoints, allowing inspection of the call stack, variable values, and program state. Variables can be monitored in real-time as the program executes, including entity IDs, component values, system state, and signatures.

12.3 Console Output

Standard output (`std::cout`) is used throughout the engine for runtime information. Developers can add temporary debug output to trace execution flow, verify values, and understand system behavior during development.

12.4 ImGui Debug Inspector

The ImGui-based editor provides real-time visual debugging.

Entity Inspector: Displays all active entities with their current component compositions. Developers can select an entity and view all its component data in a formatted panel.

Component Value Monitoring: All component values are displayed and updated in real-time. For example, Transform shows Position (x, y), Rotation, Scale. Health shows Current HP, Max HP.

Live Component Editing: Component values can be modified directly in the inspector while the game runs, enabling rapid iteration and tuning without recompilation.

System Statistics: The inspector shows which systems are active and how many entities each system is currently processing.

12.5 Performance Profiling

Basic performance metrics are available. Frame Time Display shows current FPS and frame time (ms) in the editor window. System Update Times can be measured to identify performance bottlenecks.

12.6 Debug Control Toggles

Debug features can be enabled or disabled. Debug Drawing allows collision bounds (AABB rectangles) to be rendered over sprites to visualize collision detection. Console Output Levels can be adjusted - minimal logging for release builds, detailed logging for debugging. ImGui Editor Toggle allows the editor interface to be hidden during gameplay testing (F1 key toggle) to see the game without UI overlay.

These debugging tools work together to provide comprehensive visibility into engine operation, making it easier to identify and fix issues during development.

13. Tools - Software

13.1 Development Environment

Microsoft Visual Studio 2022: Primary IDE for C++ development, debugging, and project management. Required Workloads include Desktop development with C++, .NET desktop development, and C++/CLI support for v143 build tools.

CMake (Version 3.15 or higher): Cross-platform build system generator. Generates Visual Studio solution files from CMakeLists.txt, manages project structure, external dependencies, and build configurations.

13.2 Graphics and Rendering

OpenGL 4.5: Core graphics API for 2D rendering used for sprite rendering, texture management, and shader compilation.

GLFW 3.3: Window creation and input handling. Creates OpenGL context, manages window lifecycle, processes keyboard/mouse input.

GLEW (OpenGL Extension Wrangler): Loads OpenGL extensions and provides access to modern OpenGL functions.

GLM (OpenGL Mathematics): Mathematics library for graphics used for vector and matrix operations, transformations, and projection calculations.

stb_image: Image loading library that loads PNG, JPG, BMP texture files into memory. Header-only library integrated directly into the engine.

13.3 User Interface

Dear ImGui 1.89: Immediate-mode GUI library for the editor. Provides level editor interface, entity inspector, asset browser, and debug panels for scene editing, component manipulation, and runtime debugging.

13.4 Data Serialization

RapidJSON: Fast JSON parsing and serialization library used for scene file loading and saving, entity prefab definitions, configuration files, and asset metadata. Chosen for performance and ease of integration.

13.5 Audio

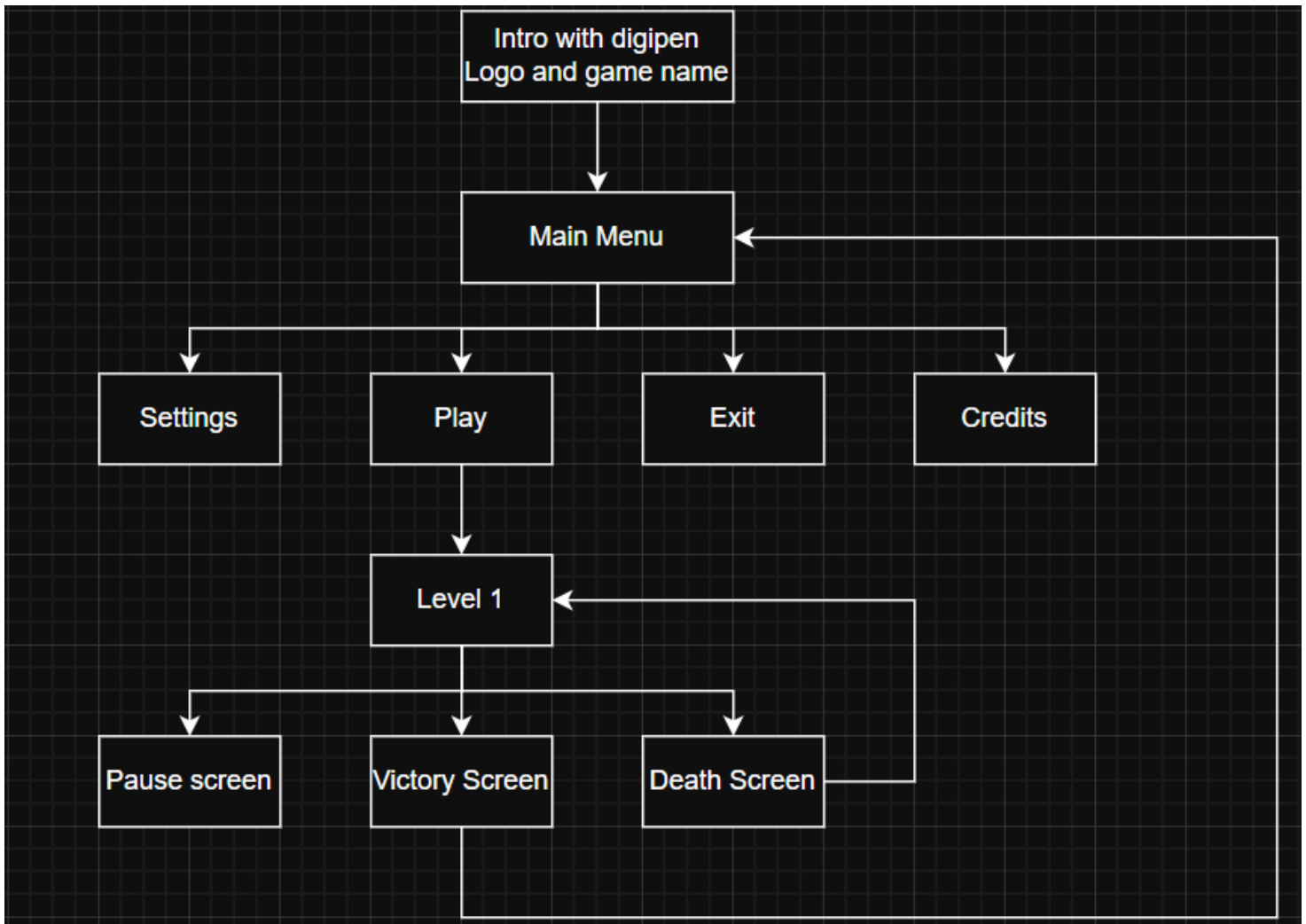
FMOD Core API or OpenAL: Audio playback and management for music streaming, sound effect playback, and 3D audio positioning. Handles audio mixing, volume control, and format support.

13.6 Scripting Runtime

.NET 8.0 Runtime: Executes C# game scripts. Components include CoreCLR (Common Language Runtime that executes managed code) and System libraries (Base .NET functionality). Downloaded as binaries (184 DLL files) and included in `extern/dotnet/bin/`.

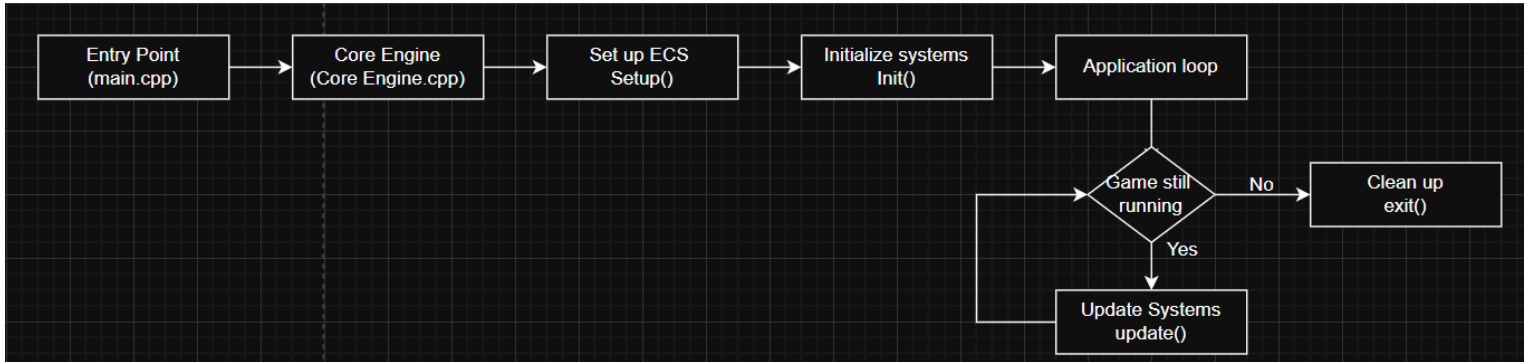
C++/CLI Bridge: Enables interoperability between C++ engine and C# scripts. Built-in Microsoft Visual Studio feature that compiles to mixed-mode assemblies containing both native and managed code.

Appendix A: Interface Flow



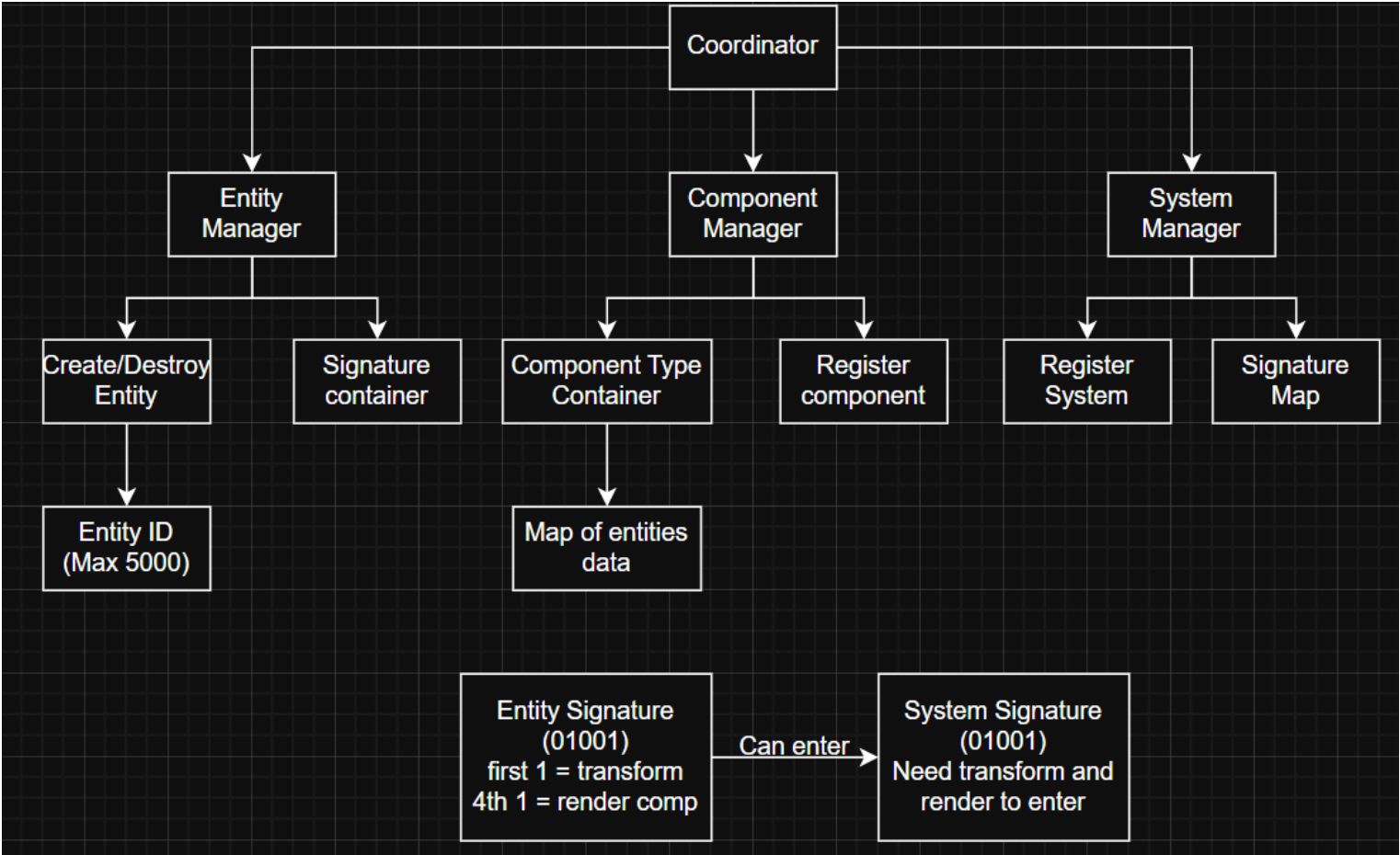
1. Intro/ Splash screen: Just play the digipen logo and show our team and game name
2. Main menu
 - > Play: Load the first level
 - > Setting: menu on controls
 - > Credits: Our team name, members and Digipen copyright
 - > Exit: Close game
3. Level 1
 - > Pause: Pressing esc, will show the menu for save game and settings
 - > Victory Screen: Show your score once game end & bring you back to home page
 - > Death screen: Shows your score and offers to let you try again once you lose

Appendix B: Engine *Architecture Flow*

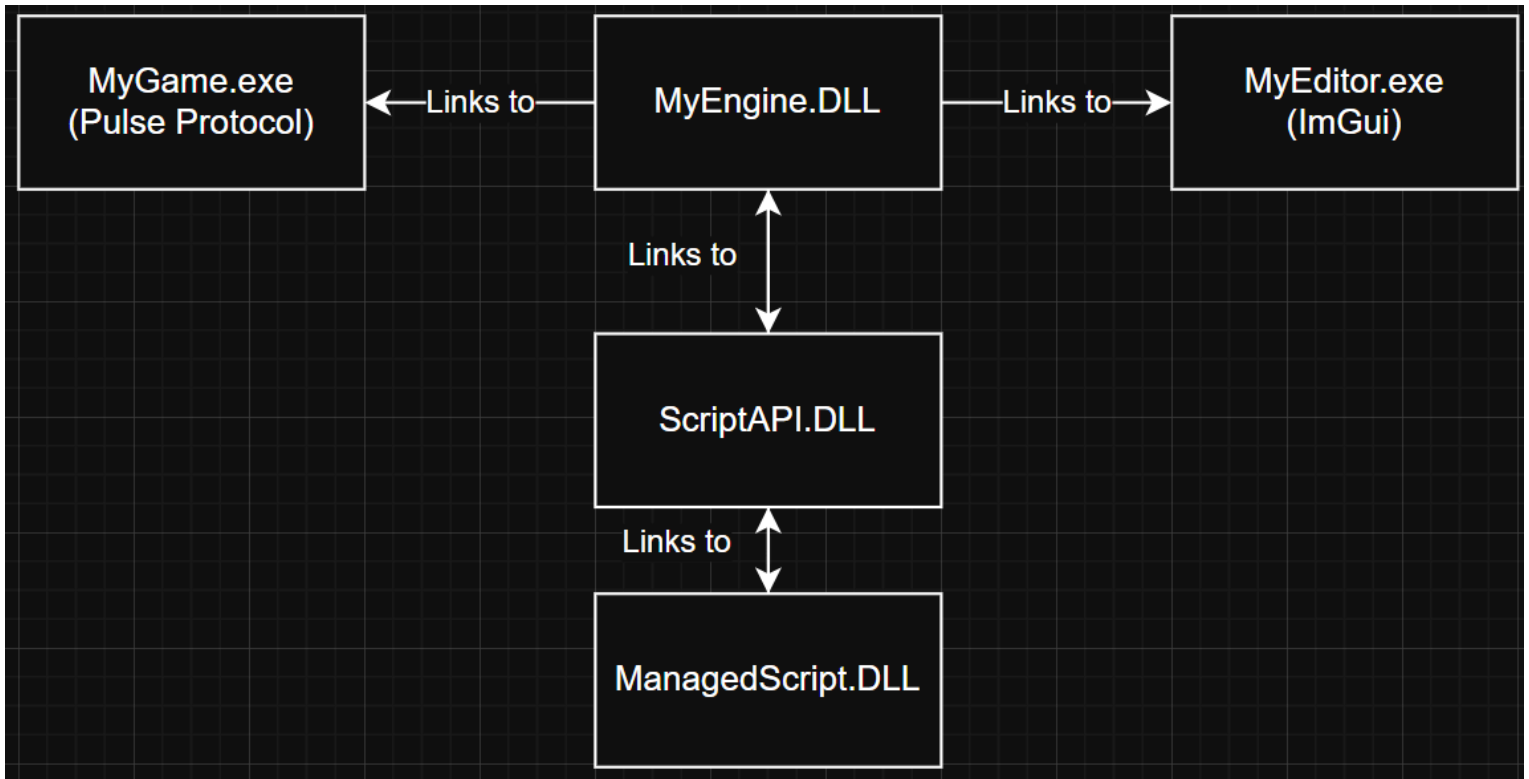


1. Entry point: In main.cpp contains the Core engine.
2. Core engine contains all of the engines setup, initialize, Application loop, updating of systems, and any cleanup.
3. Setup: The team will register component and systems that are be using here and assign signatures to each of them.
4. Initialize systems: Initialize all system such as Audio, Graphics, Collision, Input.
5. Application loop: The main game loop runs while the window is open and GameRunning flag is true.
6. Decision:
 - > If game is still running, keep updating system
 - > If game is not running, proceed to exit()
7. Clean up: Make sure the window and engine closes smoothly

Appendix C: ECS diagram

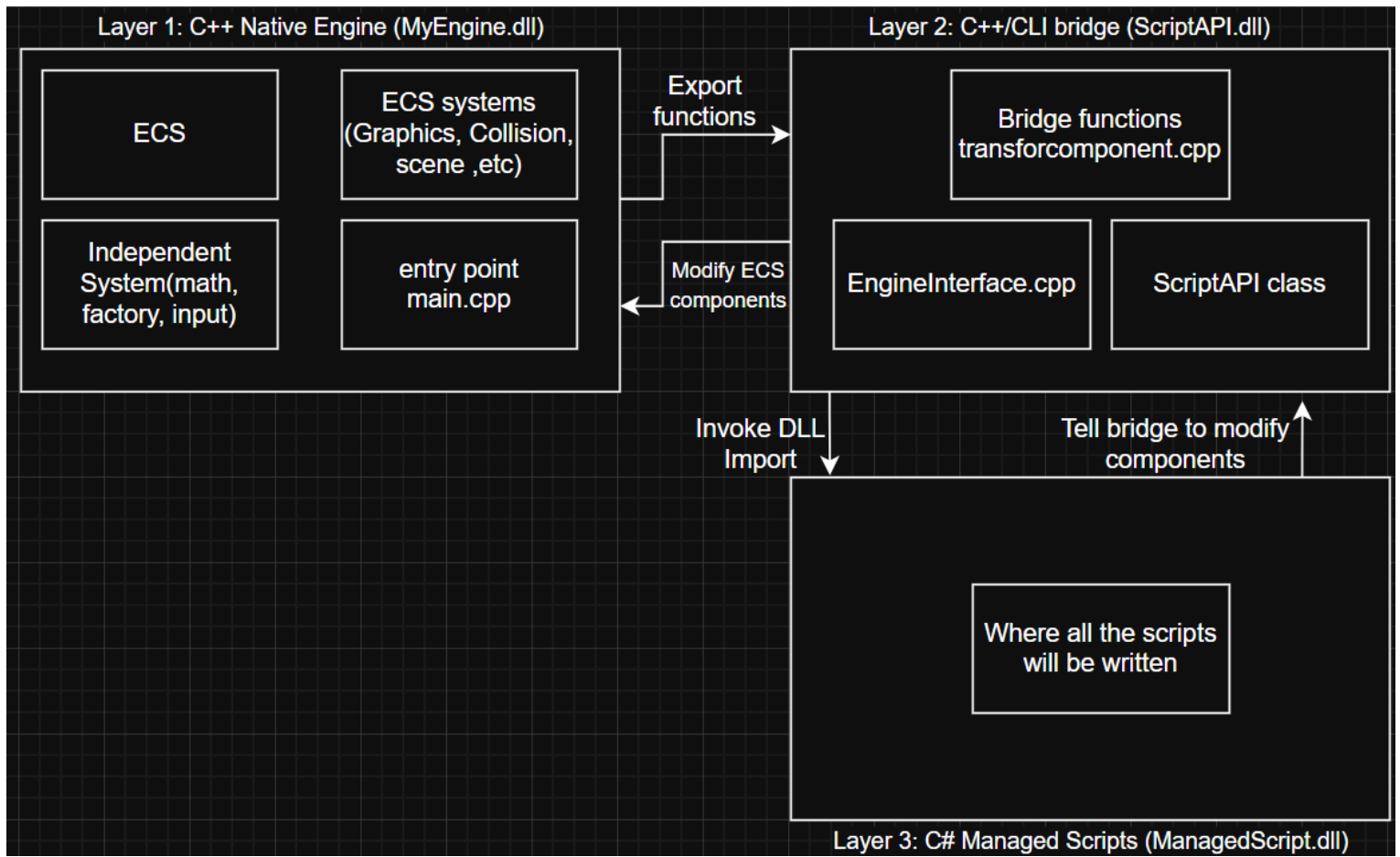


Appendix D: Project dynamic linking diagram



1. Either MyEditor or MyGame exe will be using MyEngine.DLL as its library. The difference for MyEditor and MyGame is that MyEditor has ImGui enable, which allows you to change the game like Unity)
2. Managed script is a Dynamic Linking Library as in .Net assembly, it allows for hot reloading without rebuilding the c++engine.
3. MyEngine is also linked with ScriptAPI, which in turn, is Linked with ManagedScript. This allows MyEngine and ManagedScript to “communicate” with each other.

More examples:



Appendix E: Art Requirement

1. File Naming Convention:

Art assets follow a naming system of the game object name, followed by a number if there are multiple objects of the same name.

- Background: Level_1.png, Level_2.png
- Character animation spritesheets: Rat_Attack, Rat_Run
- Placeholder assets: NAME_placeholder.png

2. File Formats:

Format: png

The format will remain as png unless a different format is required of an asset.

3. Art Creation Requirements

- All art assets will be created by the BFA student.
- Assets from other sources may be used as placeholders
- No external art assets will be used in our game
- Fonts must be sourced from DigiPen Libraries, Adobe fonts, or created ourselves

4. Sources of Art:

- Produced by team lead artist (Fenny)

Appendix F: Audio Requirements

1. File Naming Convention:

ALL audio files will follow a standardised naming system

- Sound Effects: SFX_Description_Number.mp3
- BGM: BGM_Description_Number.mp3
- Voice Over: VO_Character_Action_Number.mp3
- Placeholder: TEMP_ for placeholder assets

2. File Formats:

Format: .mp3 (Compressed, for final builds only)

Sample Rate: 44.1 kHz, 16-bit

Mono: For SFX & voice lines

Stereo: For music & environmental ambience

Normalisation: All files exported at safe levels (-1dB peak, no clipping)

3. Audio Creation Requirements

- All audio assets will be created by the audio designer through recording, synthesis or editing.
- DigiPen-provided audio libraries may be used where appropriate.
- Temporary programmer made sounds may be used in early prototypes but will be replaced before final submission
- No external/public domain/commercial audio sources will be allowed or used in our game.

4. Sources of Audio:

- Original Creation: Designed and produced by team audio designer (Carrie)
- DigiPen Libraries: Selected sounds (E.g. SFX_Alarm_Ambience, with credit)